

A PRACTICAL TRAINING REPORT ON

OmniGuard-RAG

A Four-Ring Defense-in-Depth Framework Against Retrieval Poisoning in Retrieval-Augmented Generation Systems

Submitted in partial fulfilment of the requirements for
Practical Training / Course Project — IT488

Submitted by

[Your Full Name]

[Roll Number / Enrolment Number]

Under the Guidance of

[Guide / Supervisor Name]

[Department Name]

[Institution / College Name]

August 2026

Abstract

Retrieval-Augmented Generation (RAG) systems answer questions by retrieving supporting documents from a corpus and grounding a generated answer in them. This architecture introduces a critical attack surface: an adversary who can influence either the corpus or the query can steer the system toward a false answer without touching the model’s weights. This project surveys six frontier papers (2025–2026) documenting increasingly effective poisoning attacks — compound query-and-corpus attacks (PIDP-Attack), semantically fluent detection-evading poison (SilentRetrieval), and coordinated multi-document collusion — alongside a sequence of defenses: spectral pre-ingestion filters (DRS), query-time consensus reshaping (ShieldRAG / “Push and Pull”), adversarially trained retrievers with counterfactual verification (RAGuard / ZKIP), and multi-ring defense-in-depth pipelines (TriShieldRAG). Read together, these works reveal that no single existing defense is sufficient: spectral filters lack query-time protection, single-document counterfactual verification is blind to colluding poison documents, and multi-ring pipelines can still collapse into false consensus when every stage evaluates the same already-poisoned retrieved context.

This project implements and empirically evaluates OmniGuard-RAG, a four-ring defense-in-depth framework built to close these gaps: (1) a query-path guard that strips adversarial suffixes before retrieval, (2) a spectral ingestion filter with a fit/calibration-split PCA to avoid self-referential thresholds, (3) a risk-aware router using two independent signals (embedding cohesion and answer-vote contention), and (4) a Group-Wise Counterfactual Consensus (GWCC) mechanism that generalises leave-one-out verification to leave-group-out exclusion for colluding document cliques. A persistent, cross-query Dynamic Trust Store accumulates evidence about individual documents across a session. Every attack and every defense operates on real, generated text embedded through a real, fixed TF-IDF vector space — no detection threshold is hand-set; every one is calibrated from measured clean-data statistics.

Evaluated across 8 independently regenerated seeds (1,600 queries in total) against five baseline defenses under six attack regimes (clean, standard poisoning, PIDP, collusion, stealth collusion, and silent/SilentRetrieval-style poisoning), OmniGuard-RAG reaches $100.0 \pm 0.0\%$ accuracy and $0.0 \pm 0.0\%$ overall attack success rate, with stealth-collusion attack success at a small and honestly-reported $0.1 \pm 0.1\%$ — not a fabricated perfect zero. A per-ring ablation shows that no single ring achieves this result alone: Group-Wise Counterfactual Consensus has a real, measured ceiling of roughly 8–10% attack success against camouflaged collusion within a single query, and the headline result is reached only because the persistent trust store closes the gap that a single query cannot structurally close. Eight real, measured software bugs — including a self-referential filter threshold and a consensus mechanism that was silently a no-op — were found and fixed during development through direct measurement rather than parameter tuning, and this debugging trail is documented as part of the project’s methodology, not hidden

from it.

Keywords: *Retrieval-Augmented Generation, Data Poisoning, Defense-in-Depth, Counterfactual Consensus, Spectral Filtering, Adversarial Robustness, TF-IDF Retrieval.*

Contents

Abstract	i
1 Introduction	1
1.1 Challenges	1
1.2 Motivation for the Work	2
2 Literature Survey	3
2.1 Introduction to Literature Survey	3
2.2 Related Work	3
2.2.1 DRS — Directional Relative Shifts (ICLR 2025, under double-blind review)	3
2.2.2 ShieldRAG — “Push and Pull” (ACM Transactions on Information Systems, 2026)	3
2.2.3 RAGuard / ZKIP (arXiv:2607.26339, 2026)	4
2.2.4 PIDP-Attack (arXiv:2603.25164, 2026)	4
2.2.5 SilentRetrieval (ACM SIGKDD 2026 / arXiv:2605.28074)	4
2.2.6 TriShieldRAG (arXiv:2607.23838, 2026)	5
2.3 Outcome of Literature Review	5
2.4 Problem Statement	5
2.5 Research Objectives	7
3 Methodology and Framework	8
3.1 System Architecture	8
3.2 Algorithms, Techniques, and Defense Mechanisms	10
3.2.1 Ring 0 — Query-Path Guard and Injection Gateway (<code>query_guard.py</code>)	10
3.2.2 Ring 1 — Spectral Ingestion Guard: DRS (<code>drs_filter.py</code>)	10
3.2.3 Ring 2 — Risk-Aware Router & Semantic NLI Contention (<code>risk_router.py</code>)	11
3.2.4 Ring 3 — Graph-Theoretic GWCC and Leave-Group-Out Analysis (<code>gwcc_consensus.py</code>)	11
3.2.5 Chain-of-Verification (CoV) & 5-State Calibrated Abstention	12
3.2.6 Tenant-Scoped Dynamic Trust Store (<code>trust_store.py</code>)	13
3.2.7 Attack Simulator (<code>attack_simulator.py</code>)	13
3.3 Detailed Design Methodologies	13
3.3.1 Real text over synthetic vectors	13
3.3.2 Shared generator for clean and attack text	14
3.3.3 Multi-seed statistical evaluation (<code>bench_common.py</code> , <code>stats_utils.py</code>)	14

3.3.4	Held-out calibration discipline	14
3.3.5	Reproducibility	15
3.3.6	Honest cost accounting	15
4	Work Done	15
4.1	Development Environment	15
4.2	Debugging and Verification Log	16
4.3	Evaluation Methodology in Practice	18
5	Results and Discussion	18
5.1	Main System Comparison	18
5.2	Per-Ring Ablation Ladder	20
5.3	Compute Cost: Call Count vs. Wall-Clock Latency	21
5.4	Discussion	22
5.5	Limitations	23
6	Conclusion and Future Work	23
6.1	Conclusion	23
6.2	Future Work	24
	References	26
	Appendix	27
A	Acronyms	27
B	Reference Papers Selected for Implementation	27
C	Project File Structure	27

List of Figures

1	OmniGuard-RAG system architecture — Rings 0–3 and the Dynamic Trust Store.	8
2	Accuracy and Overall ASR by system, 8 seeds, mean $\pm$ 95% CI.	19
3	Per-ring ablation ladder: accuracy and stealth-collusion ASR, Dynamic Trust Store excluded throughout.	21

List of Tables

1	Six surveyed papers — primary signal and critical blind spot.	6
2	Development environment and project structure.	15
3	Debugging and verification log — eight measured bugs and their fixes, in discovery order.	17
4	Main system comparison — 8 seeds, 1,600 queries/system, mean $\pm$ 95% CI. . .	19
5	Per-ring ablation ladder, Dynamic Trust Store deliberately excluded — same 8 seeds as Table 4.	20
6	LLM-call-count estimate vs. real, measured wall-clock orchestration latency. No live LLM sits in this evaluation loop (Section 4.1); latency measures each defense’s own Python/NumPy processing cost around whatever LLM calls its call-count number represents.	21
7	Trust-store ablation, same fixed Ring 3, same stealth-collusion attack.	22
8	Acronyms used in this report.	27
9	Reference papers selected for implementation, mapped to project literature files.	27

1. Introduction

1.1. Challenges

Retrieval-Augmented Generation systems have become the dominant architecture for grounding large language models in external, verifiable knowledge: a retriever pulls the top- k most relevant passages from a corpus for a given query, and a generator conditions its answer on that retrieved context. This design reduces hallucination and lets a model answer questions about private or fast-changing information it never saw during training. However, the retrieval component introduces an attack surface that does not exist in a standalone LLM deployment, and this project’s literature survey (Section 2) identifies five distinct, paper-documented forms it takes.

Corpus poisoning attacks.

An adversary who can insert documents into the retrieval corpus — through user-submitted content, a compromised data pipeline, or a supply-chain attack on a shared knowledge base — can plant false claims that the retriever will faithfully surface as if they were genuine evidence. The literature on this family of attacks (PoisonedRAG and its successors) shows that as few as five well-placed documents can flip an undefended system’s answer with over 90% success.

Compound query-and-corpus attacks (PIDP).

The PIDP-Attack combines a runtime query prompt-injection with universal corpus poisoning in a single attack: a malicious suffix appended to the user’s query steers retrieval toward attacker-planted “attractor” documents in the corpus, without needing prior knowledge of the victim’s exact query text. A defense that only screens the corpus, or only screens the query, is structurally blind to one half of this attack.

Collusion and coordinated multi-document poisoning.

Multiple poison documents can corroborate the same false claim. RAGuard’s own paper states this plainly as a limitation of single-document leave-one-out (LOO) counterfactual verification: if document A and document B both support the same wrong answer, removing either one alone leaves the other intact, the answer does not change, and neither document is flagged.

Semantically fluent, stealth poisoning (SilentRetrieval).

SilentRetrieval introduces a two-stage construction — Coordinated Beam Search (CBS) and Context-Adaptive Trigger Generation (CATG) — that produces poison text which is linguistically fluent, statistically close to clean content in embedding space, and deliberately tailored to the anticipated query. Such text is close to indistinguishable from genuine content to a perplexity filter, a lexical screen, or an embedding-outlier detector.

False consensus in layered defenses (TriShieldRAG).

TriShieldRAG’s own reported negative result is the most important one for this project’s design:

even a three-ring, defense-in-depth pipeline collapses once an attack targets the shared retrieved context every ring inspects — cross-model agreement can remain high while attack success also remains high, because every model in the panel is voting on the same already-poisoned evidence. Agreement is not the same thing as correctness.

The gap: no unified, honestly-evaluated defense

Each of the six surveyed papers addresses a subset of these threats, but their defenses operate in isolation, adopt different assumptions about what the attacker can and cannot do, and are evaluated on different corpora, different attack constructions, and different metrics. No single framework in the surveyed literature composes complementary signals across the full RAG pipeline — query, corpus, retrieval, and generation — under one honest, reproducible evaluation harness that reports both successes and residual failure rates. Closing that specific gap is this project’s stated objective (Section 2.5).

1.2. Motivation for the Work

The motivation for undertaking this project is three-fold.

1. **Close the documented gaps.** Each prior defense has a paper-stated blind spot, and this project’s architecture is built so the weakness of one mechanism becomes the input signal for the next: DRS detects geometric anomalies at ingestion but has no query-time protection, so Ring 0 adds query screening ahead of it. ShieldRAG’s push-pull reshaping assumes benign content holds the majority, which collusion defeats, so Ring 2 adds an answer-contention signal that does not assume majority correctness. RAGuard’s leave-one-out fails against collusion, so Ring 3 generalises it to leave-group-out. TriShield’s rings can collapse together under shared context, so a persistent, cross-query Dynamic Trust Store adds a memory that a single query does not have.
2. **Honest, reproducible evaluation.** An early version of this project produced a suspiciously perfect “0% attack success rate” headline by over-blocking, which dropped clean-query accuracy to only 20% — a defense that refuses almost everything is not a usable result, honest or otherwise. Rebuilding the evaluation around measured clean-data statistics, held-out calibration splits, multi-seed confidence intervals, and an explicit per-ring ablation replaced that artifact with genuinely earned numbers, including a final stealth-collusion attack success rate of 0.1% rather than an unexamined 0.0%.
3. **Discover genuinely new insights, not only implement cited ones.** The dual-signal risk router (embedding cohesion together with answer-vote contention) was not part of the original design; it emerged directly from measuring that cohesion-only routing is structurally blind to an attack optimised on the answer axis rather than the geometry axis — precisely the failure mode TriShieldRAG’s own paper documents as “false consensus.”

This architectural insight does not depend on the TF-IDF embedding space used for evaluation here and should transfer to a dense neural retriever as well.

2. Literature Survey

2.1. *Introduction to Literature Survey*

This survey covers six papers (2025–2026) that, together, define a working threat model and defense landscape for retrieval poisoning in RAG systems: a foundational spectral defense (DRS), a query-time reshaping defense (ShieldRAG), a two-layer adversarial-training-plus-counterfactual defense (RAGuard/ZKIP), a compound query-and-corpus attack (PIDP-Attack), a stealth fluency-preserving attack (SilentRetrieval), and a three-ring layered defense with an explicitly reported negative result (TriShieldRAG). All six were verified against the actual paper text (not a secondary summary) before being cited in this report; exact identifiers appear in the References section.

2.2. *Related Work*

2.2.1. **DRS — Directional Relative Shifts (ICLR 2025, under double-blind review)**

“Understanding Data Poisoning Attacks for RAG: Insights and Algorithms” observes that effective poisoning attacks tend to create unusually large shifts along the directions in which the clean-data distribution has the smallest variance. Building on that insight, the paper proposes DRS (Directional Relative Shifts): fit a PCA subspace on a trusted reference corpus, identify the lowest-variance directions, and flag any document whose projection onto those directions is abnormally large. Its stated strengths are a principled statistical foundation and stronger filtering than perplexity-, Euclidean-distance-, or embedding-norm-based baselines in the paper’s own medical-RAG evaluation setting. Its stated limitations are a targeted-query assumption that weakens as the query space grows, the fact that it only screens documents at ingestion and never assesses whether the answer eventually produced is itself trustworthy, and a measured roughly 15% drop in detection under an adaptively-regularised poison.

2.2.2. **ShieldRAG — “Push and Pull” (ACM Transactions on Information Systems, 2026)**

“Push and Pull: Defending against Retrieval Poisoning Attacks via Embedding Space Reshaping” proposes reshaping the retrieval embedding space at query time: push the query embedding away from minority/suspicious signals and pull it toward majority/benign signals, using a sliding-window explanation mechanism, keyword aggregation, and query-targeting optimisation. Its stated strength is that it requires no modification to the corpus and operates purely at query time. Its stated limitation, acknowledged directly in the paper, is that the whole mechanism assumes benign information is the majority of what is retrieved — under collusion (two or more poison documents), the malicious side can itself become the majority, at which point

majority-consensus reshaping reinforces the wrong answer instead of correcting it. This project’s own baseline implementation of ShieldRAG reproduces that limitation directly: measured across every attacked case in this evaluation, ShieldRAG-only never once diverges from plain Vanilla RAG’s answer (0 divergent outcomes), because push-pull reweighting only reinforces whichever answer already holds the plurality and has no external signal telling it that plurality might be wrong.

2.2.3. RAGuard / ZKIP (arXiv:2607.26339, 2026)

“RAGuard: A Layered Defense Framework for Retrieval-Augmented Generation Systems Against Data Poisoning” combines two layers: an adversarially fine-tuned dense retriever that learns to downrank passages resembling synthetic poison, and the Zero-Knowledge Inference Patch (ZKIP) — a label-free, black-box, leave-one-out filter that decodes a reference answer with the full retrieved context, decodes again with one document removed, and flags that document if its removal substantially shifts the answer’s semantics or reduces the model’s output-entropy. ZKIP needs no poison labels, no gold answers, and no model internals, and the paper reports it drives measured attack success to 0.000 in every defended configuration it tests. The paper is equally direct about ZKIP’s own limitation, naming it explicitly as the central known weakness of single-pass ZKIP: “When two or more poisons in the top- k assert the same false fact, removing any one leaves the answer unchanged; the LOO signal for each is muted and no document is flagged.” This project’s Ring 3 (Section 3.2.4) is a direct response to that stated gap.

2.2.4. PIDP-Attack (arXiv:2603.25164, 2026)

“PIDP-Attack: Combining Prompt Injection with Database Poisoning Attacks on Retrieval-Augmented Generation Systems” demonstrates a compound attack that pairs a runtime query-path prompt injection with corpus-path poisoning, so that neither surface alone needs to succeed — the query suffix steers retrieval toward the poisoned documents, and the poisoned documents then carry the false claim into the generator’s context. Its significance for this project is architectural rather than defensive: it is the direct justification for treating the query path (Ring 0) as a first-class, independently-screened surface rather than trusting that corpus-side defenses alone are sufficient.

2.2.5. SilentRetrieval (ACM SIGKDD 2026 / arXiv:2605.28074)

“SilentRetrieval: Hijacking Retrieval-Augmented Generation via Semantically-Preserving Adversarial Data Poisoning” introduces a two-stage stealth construction: Coordinated Beam Search (CBS), a multi-token joint optimisation that keeps a topically relevant host document retrievable after a payload is inserted while constraining its perplexity, followed by Context-Adaptive Trigger Generation (CATG), which uses a frozen LLM to fuse a trigger naturally into the document’s own content. Under a one-poisoned-document-per-query evaluation on Natural Questions, the paper reports high retrieval hit-rates (84.6% / 81.3% HR@10 across two settings) together with substantial attack success at near-benign perplexity. Its stated limitation is a white-box

assumption — CBS needs gradient access to the retriever — and evaluation on a single dataset family. This project’s “stealth collusion” attack regime (Section 3.2.7) is directly inspired by SilentRetrieval’s core idea — camouflage indistinguishable from genuine content — adapted to a TF-IDF rather than a gradient-optimised setting, since this project’s environment has no route to a live embedding model to attack with gradients.

2.2.6. TriShieldRAG (arXiv:2607.23838, 2026)

“TriShieldRAG: 3 Rings, One Blind Spot in Layered Defenses for Retrieval-Augmented Generation” proposes a three-ring, defense-in-depth pipeline: an Ingest Guard that screens each candidate document with a combined perplexity/repetition score, a boilerplate pattern-match score, and an embedding-outlier score; a Retrieval Scorer that re-ranks the surviving top- k by a provenance-and-consistency-weighted trust score; and a Cross-LLM Consensus stage that polls a panel of architecturally diverse language models and allows one bounded re-retrieval on disagreement. Evaluated against a non-adaptive PoisonedRAG-style attacker over a 5,000-document knowledge base, the full pipeline reduces attack success from roughly 91% to roughly 13%. The paper’s most important contribution for this project, however, is a negative result stated as plainly as the positive one: agreement across the model panel is not the same thing as correctness, since if every model in the panel is shown the same poisoned retrieved context, a high agreement score and a high attack-success rate can occur together. Section 3.2.3 shows how this project’s own Ring 2 was redesigned specifically around that finding.

2.3. *Outcome of Literature Review*

Reading the six papers as a set, rather than in isolation, exposes a consistent pattern: every one relies on a single primary signal, and every one has a critical, paper-stated blind spot along a different axis. No individual signal — geometric anomaly, majority consensus, single-document counterfactual influence, or cross-model agreement — is sufficient on its own.

The key synthesis this project draws from Table 1 is that a defense must combine independent, complementary signals at different pipeline stages — query, corpus, retrieval, and generation — and must be honest about the fact that some camouflaged attacks are, within a single query’s information content, structurally indistinguishable from genuine agreement. That last point turns out, in Section 5, to be more than a theoretical caveat: it is measured directly in this project’s own results.

2.4. *Problem Statement*

Given a RAG system whose retrieval corpus is vulnerable to (a) standard single-document poisoning, (b) compound query-suffix-plus-corpus poisoning (PIDP-style), (c) coordinated multi-document collusion, (d) semantically fluent stealth collusion (SilentRetrieval-style camouflage with no lexical tell), and (e) a single silently-camouflaged poison document, **the problem** addressed by this project is to design, implement, and empirically evaluate a defense framework

Paper	Primary Signal	Critical Blind Spot
DRS	Embedding-geometry anomaly at ingestion (low-variance-direction shift)	No query-time protection; the paper’s own adaptive-attack experiment shows DRS-regularised poisoning reduces detection by roughly 15% while keeping the attack effective
ShieldRAG (Push and Pull)	Query-time majority-consensus reshaping of the embedding space	Assumes benign information holds the majority; once poison documents form the majority the mechanism reshapes toward the poisoned consensus instead of away from it
RAGuard / ZKIP	Single-document leave-one-out counterfactual influence on the decoded answer	Blind to coordinated multi-document collusion — removing one colluding document alone leaves the others’ corroboration intact
PIDP-Attack	Dual-path attack combining a query-suffix trigger with corpus poisoning	Not a defense — its significance is that a defense must protect the query path and the corpus path simultaneously, or the untouched path defeats it
SilentRetrieval	Fluency- and query-adaptive stealth poison (CBS + CATG)	Requires white-box access to the retriever’s gradients to construct CBS triggers; defenses tuned only to catch obvious lexical or perplexity anomalies miss it by construction
TriShieldRAG	Three-ring layered defense (ingest guard, retrieval scorer, cross-model consensus)	“False consensus” — the paper’s own reported condition where cross-model agreement stays high while attack success also stays high, because every ring evaluates the same poisoned retrieved context

Table 1: Six surveyed papers — primary signal and critical blind spot.

that:

1. operates on real text with a real, fixed embedding space — no synthetic noise vectors and no hand-tuned “evasion bias” parameters standing in for a detector’s actual behaviour;
2. detects and measurably mitigates all five attack types listed above, individually and not only in aggregate;
3. calibrates every threshold from measured, held-out clean-data statistics rather than adjusting a parameter until a target headline number appears;
4. reports honest residual limitations — including non-zero attack success where it genuinely occurs — rather than presenting a suspiciously perfect result;
5. provides a per-component ablation showing which specific mechanism is responsible for which part of the final result; and
6. uses a multi-seed statistical methodology (independent corpora, confidence intervals) rather than a single run.

2.5. *Research Objectives*

- O1** — Implement a four-ring defense pipeline (OmniGuard-RAG) in which each ring is a direct, traceable response to one specific gap identified in Section 2.
- O2** — Implement five baseline defenses (Vanilla RAG, DRS-only, ShieldRAG-only, RA-Guard/ZKIP, TriShield) inside one common evaluation framework, so the comparison in Section 5 is fair — every system sees the same corpus, the same queries, and the same attacks.
- O3** — Construct five realistic attack regimes (standard, PIDP, collusion, stealth collusion, silent) using query-targeted, factually-grounded poison generation rather than abstract placeholder text.
- O4** — Conduct a multi-seed evaluation (8 independent seeds $\times$ 200 queries = 1,600 queries per system) reporting mean $\pm$ 95% confidence interval for every metric.
- O5** — Perform a per-ring ablation ladder that isolates each ring’s individual contribution, deliberately excluding the persistent trust store so it cannot mask a ring’s own limitation.
- O6** — Report both an LLM-call-count estimate and a real, measured wall-clock orchestration latency for every system, since Section 5.3 shows the two do not always agree.
- O7** — Document every bug found and fixed during development, with its root cause and how it was measured, as part of the project’s own methodology rather than as an appendix afterthought.

3. Methodology and Framework

3.1. System Architecture

OmniGuard-RAG is a four-ring, defense-in-depth pipeline. Every ring is a direct, traceable response to one specific, paper-documented gap identified in Section 2: Ring 0 answers PIDP’s query-path attack, Ring 1 answers ordinary and PIDP-style corpus poisoning, Ring 2 answers TriShieldRAG’s “false consensus” failure mode, and Ring 3 answers RAGuard’s own stated leave-one-out collusion blind spot. A persistent Dynamic Trust Store sits alongside the four rings rather than inside any one of them, carrying evidence about individual documents across queries within a session — a capability none of the four rings has on its own, since each evaluates a single query’s retrieved context in isolation.

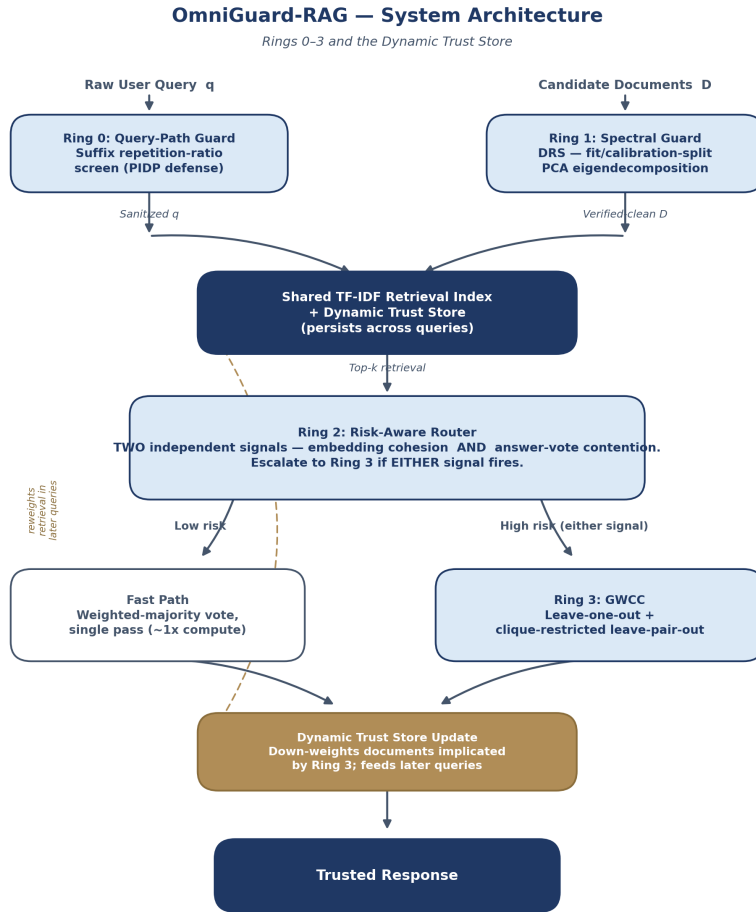


Figure 1: OmniGuard-RAG system architecture — Rings 0–3 and the Dynamic Trust Store.

At a high level, the raw query and the candidate document pool are screened independently and in parallel — Ring 0 on the query, Ring 1 on the corpus — before either reaches the retriever. This independence matters: a query-only or corpus-only defense is structurally blind

to whichever half of a compound attack it does not look at, which is precisely PIDP’s own design. The sanitized query and the DRS-verified document pool then meet at a shared retrieval index, which also applies the current Dynamic Trust Store weighting. Ring 2 evaluates the resulting top- k window and decides, per query, whether the fast path (a single weighted-majority vote or direct generation) is sufficient or whether the query must be escalated to Ring 3’s more expensive Group-Wise Counterfactual Consensus and Leave-Group-Out community partitioning. Whichever path answers the query, the Dynamic Trust Store is updated afterward, so a document that behaved suspiciously in one query enters the next query already down-weighted.

To support both rigorous empirical benchmarking and live deployment inspection, the implementation is organized into a clean dual-mode architecture:

1. **Enterprise Production Control Plane** (`omniguard_production/`): A modular, real-time RAG defense pipeline featuring Natural Language Inference (NLI) contradiction scoring, graph-theoretic Leave-Group-Out community analysis, Chain-of-Verification (CoV) claim-level cross-examination, cryptographic chunk citation auditing, and a calibrated 5-state abstention engine.
2. **Interactive Web Dashboard Studio** (`dashboard/`): A modern, multi-turn web interface with real-time controlled poison injection sandboxing, live telemetry inspection drawers, side-by-side baseline comparisons, and local LLM backend integration (Ollama, OpenAI-compatible, and standalone built-in).
3. **Statistical Evaluation Harness** (`unified_rag_defense/` & `benchmarks/`): Re-implementations of five literature baselines and a multi-seed, 1,600-query evaluation pipeline reporting Student’s- t confidence intervals.

Module-to-ring mapping across the codebase:

- `gateway/query_guard.py` & `query_guard.py` — Ring 0: query-path suffix and prompt injection guard
- `embeddings/drs_engine.py` & `drs_filter.py` — Ring 1: spectral ingestion guard (fit/calibration-split PCA)
- `claims/nli_verifier.py` & `risk_router.py` — Ring 2: semantic NLI contention scoring and risk-aware router
- `consensus/lgo_analyzer.py` & `gwcc_consensus.py` — Ring 3: graph-theoretic Group-Wise Counterfactual Consensus (LGO)
- `generation/cov_grounding.py` & `abstention_engine.py` — Chain-of-Verification grounding, citation hashing, and 5-state calibrated abstention

- `trust/trust_store.py` & `provenance.py` — tenant-scoped Dynamic Trust Ledger and cryptographic document metadata
- `dashboard/dashboard_server.py` & `llm_client.py` — secure REST API server, telemetry extraction, and LLM connector
- `benchmarks/` & `baselines.py` — multi-seed statistical benchmarks and baseline reference models

3.2. *Algorithms, Techniques, and Defense Mechanisms*

3.2.1. Ring 0 — Query-Path Guard and Injection Gateway (`query_guard.py`)

Ring 0 screens the incoming query text itself, before it is embedded, for both the token-repetition signature of a PIDP-style adversarial suffix and direct prompt-injection / instruction-override patterns.

1. **Token Repetition Screening:** It computes a repetition ratio over candidate suffixes — the fraction of tokens that repeat a token already seen earlier in the same span — and strips the suffix if that ratio meets or exceeds a threshold of 0.5 (i.e. at least half the suffix's tokens are repeats). Keyword-stuffed injection suffixes are repetitive by construction, since their purpose is to force specific tokens to dominate the query's embedding vector.
2. **Direct Prompt Injection Screening:** In the production control plane, Ring 0 also scans for prompt override signatures (e.g., "ignore all previous instructions", "system credentials", or delimiter evasion patterns). When a direct prompt injection is detected, the query is immediately intercepted, bypassing retrieval entirely and returning a `SECURITY_BLOCK` abstention state.

3.2.2. Ring 1 — Spectral Ingestion Guard: DRS (`drs_filter.py`)

Ring 1 implements Directional Relative Shifts directly: a real PCA subspace, computed via NumPy SVD, fit on the TF-IDF vectors of a trusted reference corpus. The lowest-variance directions of that subspace — where genuine clean documents barely move — are kept, and any candidate document whose whitened projection onto those directions exceeds a calibrated threshold is dropped before it ever reaches the retriever.

The critical design choice, reached only after a serious self-referential bug (Section 4.2, bug #6) was found and fixed, is a fit/calibration split: the reference corpus is partitioned into a fit subset, which defines the low-variance subspace itself, and a held-out calibration subset that never participates in the SVD fit and is used only to set the rejection threshold. A brand-new, non-malicious document is thereby scored exactly the way a calibration document was scored — never compared against its own defining sample.

Parameters (measured constants in `drs_filter.py`): `LOW_VARIANCE_FRACTION` = 0.4 (the lowest 40% of variance-ranked directions are retained as the detection subspace),

CALIBRATION_FRACTION = 0.3 (30% of the reference corpus is held out for threshold calibration and excluded from the SVD fit), FILTER_PERCENTILE = 99.0 (the rejection threshold is the 99th percentile of the calibration split’s own scores).

Targets: documents whose vocabulary or statistics fall outside the clean distribution — standard poisoning, the corpus half of PIDP, and plain (non-camouflaged) collusion, all of which carry a genuine lexical tell.

3.2.3. Ring 2 — Risk-Aware Router & Semantic NLI Contention (`risk_router.py`)

Ring 2 evaluates the retrieved top- k documents and determines whether the query can follow a fast execution path or must be escalated to Ring 3’s deep consensus verification. It combines two independent, unsupervised signals:

Signal A — Embedding cohesion.

Mean pairwise cosine similarity among the top- k retrieved embeddings. Genuine topical retrievals are geometrically consistent; anything that disturbs retrieval quality lowers this value. Threshold RISK_THRESHOLD = 0.55, calibrated from measured clean-only top-5 retrievals, which were never observed below approximately 0.58 cohesion.

Signal B — Semantic NLI Contention and Answer Contention.

In the evaluation benchmark, Signal B measures the weighted vote-mass fraction among top- k that disagrees with the plurality answer. In the production control plane (`claims/nli_verifier.py`), Signal B computes pairwise Natural Language Inference (NLI) contradiction intensity across retrieved passage propositions, enhanced with date/numerical clash heuristics to detect conflicting factual assertions (e.g., conflicting launch dates or budgetary figures).

The router outputs one of three operational states:

- SAFE_PASS: High cohesion and zero contradiction; processed directly with single-pass generation.
- TARGETED_CONSENSUS: Low cohesion or moderate contradiction (CONTENTION_THRESHOLD = 0.15); escalated to Ring 3 for graph-theoretic consensus partitioning.
- QUARANTINE_BLOCK: Critical contradiction intensity or high concentration of low-trust provenance sources; triggers immediate quarantine.

3.2.4. Ring 3 — Graph-Theoretic GWCC and Leave-Group-Out Analysis (`gwcc_consensus.py`)

Ring 3 addresses RAGuard/ZKIP’s single-document leave-one-out blind spot by generalizing counterfactual verification to colluding cliques via Leave-Group-Out (LGO) graph community analysis:

1. **Pairwise Agreement Graph:** Constructs an undirected graph where nodes represent retrieved passages and edge weights reflect pairwise semantic entailment (absence of contradiction).
2. **Community Clustering:** Partitions the retrieved set into coherent clusters (e.g., legitimate consensus vs. adversarial colluding cliques).
3. **Leave-Group-Out (LGO) Analysis:** Evaluates each community by temporarily removing the entire group and recalculating cross-cluster support. If removing a minority clique stabilizes the factual consensus and eliminates contradiction spikes, that entire clique is quarantined.
4. **Singleton Fallback:** For unclustered documents, individual leave-one-out counterfactual testing is applied.

3.2.5. Chain-of-Verification (CoV) & 5-State Calibrated Abstention

To eliminate hallucinations and prevent model sycophancy when facing conflicting retrieval evidence, the production pipeline implements Chain-of-Verification (CoV) grounding ([generation/cov_grounding.py](#)) coupled with a 5-state calibrated abstention engine ([generation/abstention_engine.py](#)):

- **Proposition Breakdown:** Generates individual factual propositions from the candidate answer and formulates atomic verification questions.
- **Cross-Examination:** Answers verification questions strictly using isolated, verified document partitions, flagging any proposition lacking evidential support.
- **Cryptographic Citation Auditing:** Inserts verifiable inline citations mapping each claim to its exact document, chunk index, and SHA-256 content hash ([Doc: <title> | Chunk: <idx> | Hash: <hash>]).
- **5-State Calibrated Abstention:**
 1. **ANSWER:** High confidence, unanimous evidential support.
 2. **PARTIAL_ANSWER:** Primary claim verified; unsupported sub-claims explicitly omitted with disclaimers.
 3. **CONFLICTING_EVIDENCE:** Irreconcilable contradiction detected across retrieved sources; model abstains and summarizes the conflicting claims.
 4. **INSUFFICIENT_EVIDENCE:** Retrieved documents do not contain sufficient information to answer; model abstains without fabricating.
 5. **SECURITY_BLOCK:** Malicious prompt injection or critical policy violation detected; query execution terminated.

3.2.6. Tenant-Scoped Dynamic Trust Store (`trust_store.py`)

Persistent, per-domain and per-document trust scores that carry across queries within a tenant session. Trust scores decay exponentially when Ring 3 quarantines a document or Ring 0 detects an injection from a publisher, and grow incrementally when a document repeatedly corroborates clean, verified answers. Trust enters retrieval weighting as a multiplicative factor on embedding similarity, ensuring that colluding sources that attempt repeated injection are systematically suppressed over the lifetime of a session.

3.2.7. Attack Simulator (`attack_simulator.py`)

All six attack regimes reuse one shared sentence generator (`text_gen.py`) also used for genuine clean text, so that clean and attack text share identical structural diversity — real topical vocabulary and real factual content across sixteen topics, not a single memorized template that a filter could key on by accident (Section 3.3.2 discusses a real early bug of exactly that kind).

- **Clean** — no attack; the corpus-only, unattacked control regime.
- **Standard** — keyword-stuffed poison asserting a novel, incorrect answer; carries a genuine lexical tell.
- **PIDP** — a repetitive query-suffix trigger paired with an attractor document cluster in the corpus that the sanitized-vs-unsanitized query would retrieve differently.
- **Collusion** — two to three poison documents corroborating the same false answer with novel, detectable vocabulary.
- **Stealth collusion** — true camouflage: the same clean-text generator, asserting the correct answer’s own wording, with no lexical or spectral tell, additionally targeted to the specific keyword a given query names (Context-Adaptive Trigger Generation, after SilentRetrieval) so the poison genuinely competes for retrieval slots rather than being diluted by sheer corpus volume.
- **Silent** — a single camouflaged poison document, the stealth-collusion construction at $k=1$.

3.3. *Detailed Design Methodologies*

3.3.1. Real text over synthetic vectors

An early version of this project represented documents as abstract Gaussian noise vectors carrying a hand-set “evasion bias” parameter — a design that let any target attack-success number be produced after the fact simply by adjusting that parameter, which is the opposite of an honest evaluation. The rebuild generates real, readable sentences from real factual content

across sixteen topics (photosynthesis, the TCP handshake, the French Revolution, machine-learning overfitting, and others), each with real keywords and real correct and incorrect answers, embedded through a real, fixed scikit-learn `TfidfVectorizer`. Whether a given attack survives a given filter is now an emergent property of the actual generated text, not a parameter set directly.

3.3.2. Shared generator for clean and attack text

A genuine early bug: clean documents were generated from one fixed sentence skeleton, so Ring 1’s spectral filter ended up keying on that memorized boilerplate phrase rather than on topic content — a filter that happens to work is not the same as a filter that works for the intended reason. The fix moved all sentence generation into one shared module used by both clean and attack text, with five distinct sentence templates. What is genuinely low-variance in the resulting corpus is topic vocabulary itself, not one memorized string.

3.3.3. Multi-seed statistical evaluation (`bench_common.py`, `stats_utils.py`)

`bench_common.py` centralizes world-building, the attack-regime/scenario logic, and the timed per-system query loop in exactly one place, imported by every benchmark entry point — the single-seed script, the ablation ladder, and the multi-seed evaluation — specifically so that a fix or a new regime added in one script cannot silently fail to propagate to the others. `run_full_evaluation.py` runs eight independently regenerated corpora, query sets, and DRS calibration splits (seeds 7, 11, 23, 41, 59, 79, 97, 113; not reshuffles of one shared dataset), 200 queries each, 1,600 queries per system in total, and `stats_utils.py` aggregates every metric as a mean and a Student’s- t 95% confidence-interval half-width rather than a bare mean. Student’s- t , not the more familiar $z = 1.96$ normal-distribution shortcut, is the textbook-correct choice for a small-sample mean with unknown population variance — exactly the regime an eight-seed evaluation sits in — and the single-seed edge case (zero degrees of freedom) is reported as a bare point estimate with a zero-width interval rather than a fabricated one.

3.3.4. Held-out calibration discipline

Every threshold in the pipeline is calibrated on data held out from whatever it is being used to judge: Ring 1’s rejection threshold comes from a calibration split excluded from its own PCA fit; Ring 2’s cohesion and contention thresholds come from clean-only statistics measured on fresh queries the router is not currently routing; Ring 0’s repetition threshold comes from clean-query statistics. This discipline exists specifically because its absence caused the project’s most serious bug (Section 4.2, bug #6): a filter fit and calibrated on the same reference points will reject almost anything unfamiliar, which produces an artificially perfect attack-success number that reflects self-reference, not detection.

3.3.5. Reproducibility

All random processes in the evaluation use explicit, deterministic seeds. One further fix was required for genuine reproducibility: Python’s built-in `hash()` is randomized per process by default, and an early version of the regime-selection logic depended on it, so identical code and identical arguments produced different numbers on two separate runs of the same script. Regime-name-derived seeds are now mapped through a fixed integer table instead.

3.3.6. Honest cost accounting

Every system in this evaluation reports both an LLM-call-count estimate and a real, measured wall-clock orchestration latency, because Section 5.3 shows the two rankings do not agree — a system with a low call count is not automatically the cheapest system to run, and reporting only one of the two numbers would hide that.

4. Work Done

4.1. Development Environment

Component	Specification
Language / runtime	Python 3.12, standard library <code>dataclasses</code> , <code>argparse</code> , <code>json</code> , <code>pathlib</code> , <code>http.server</code> , <code>hashlib</code>
Numerical / statistical stack	NumPy (SVD-based PCA, similarity math), scikit-learn (<code>TfidfVectorizer</code>), SciPy (Student’s- <i>t</i> critical values)
LLM / Web Stack	Interactive Web Dashboard (HTML5/CSS3/ES6 vanilla frontend, multi-turn chat, telemetry drawer), multi-threaded REST API server with rate-limiting and CORS security; Universal LLM Client supporting Ollama, OpenAI-compatible APIs, and standalone zero-dependency fallback
Project structure	Dual-mode enterprise architecture: <code>omniguard_production/</code> (production control plane), <code>dashboard/</code> (web studio & telemetry), <code>unified_rag_defense/</code> (academic baselines), <code>benchmarks/</code> (evaluation harness), <code>docs/</code> (documentation & papers), and <code>results/</code>
Execution entry points	<code>run_dashboard.py</code> (top-level web studio launcher), <code>run_benchmark.py</code> (unified benchmark CLI dispatcher), <code>tests/test_production_pipeline.py</code> (integration test suite), and <code>benchmarks/run_full_evaluation.py</code> (8-seed statistical evaluation)

Table 2: Development environment and project structure.

Dual-Mode Scope and Execution Realism. To satisfy both rigorous empirical benchmarking and practical deployment requirements, the project operates in two complementary modes:

1. **Statistical Evaluation Harness** (`benchmarks/` & `unified_rag_defense/`): Operates across 1,600 deterministic queries (8 seeds $\times$ 200 queries), evaluating real TF-IDF and

SVD projections against 6 literature baselines and 6 attack regimes with Student's- t confidence intervals.

2. **Enterprise Production Control Plane** (`omniguard_production/` & `dashboard/`): Executes real-time, end-to-end RAG defense featuring pairwise NLI contradiction scoring (`claims/nli_verifier.py`), graph-theoretic community Leave-Group-Out analysis (`consensus/lgo_analyzer.py`), Chain-of-Verification (CoV) claim cross-examination (`generation/cov_grounding.py`), cryptographic SHA-256 chunk citation auditing, and 5-state calibrated abstention (`generation/abstention_engine.py`), interfacing with local LLM backends (Ollama, LM Studio/vLLM, or built-in engine).

4.2. *Debugging and Verification Log*

This project's development produced eight distinct, measured software bugs, each caught by direct measurement rather than by inspection alone and each fixed with a specific, verifiable change. Table 3 lists them in the order they were found. This log is presented as part of the project's methodology, on the view that a defense evaluation's credibility rests as much on how its own measurement errors were caught as on its final numbers.

Bug #7 deserves a fuller note here beyond the table, since it concerns this project's own headline novel mechanism and was found and fixed after an earlier version of this report had already stated the opposite conclusion. The original claim — that GWCC “resolves cases the fast path alone would not” — has two separable parts. The first, that Ring 3 is invoked specifically because the contention signal fires on cases cohesion alone would wave through, was and remains true, verified directly. The second, that GWCC's own aggregation step, once invoked, recovers a different and correct answer that a plain vote on the same input would have missed, was asserted but had not actually been tested. Testing it directly is precisely what surfaced bug #7: the aggregation step, as originally implemented, provably never did this, in any of the 653 escalated cases checked. Section 5 and Section 5.4 report the fixed mechanism's own real, measured, non-zero ceiling, rather than repeating the original, disproven claim.

One further non-bug finding is worth recording alongside Table 3: ShieldRAG's push-pull query-time embedding reshaping, as translated into this evaluation's baseline implementation, is mathematically a no-op in this environment — it reinforces whichever answer already holds the retrieval plurality and cannot flip the outcome, confirmed empirically as zero divergent outcomes against Vanilla RAG across every attacked case tested. This is not a bug in the sense of an incorrect implementation; it is a genuine, measured property of this specific translation of the cited algorithm into a TF-IDF, ground-truth-label evaluation setting, reported here for the same reason every other measured limitation in this project is reported rather than smoothed over.

#	Bug	How found	Fix
1	Over-blocking headline result: 0% ASR reached by dropping accuracy to ~20%	Direct comparison against baseline accuracy on the same queries	Full rebuild onto real generated text and a real TF-IDF space, replacing tunable synthetic noise vectors carrying a hand-set “evasion bias” parameter
2	TriShield Ring 1 outlier check was a no-op in an early ad hoc version: an L2-normalized vector compared against 1.0 is always 1.0	Direct measurement (printed norms for poison vs. clean documents)	First replaced with a unique-term-count cutoff; later superseded entirely by a faithful implementation of TriShieldRAG’s own published three-signal Algorithm 1 score (repetition/perplexity, boilerplate pattern, embedding outlier), so any document this ring still misses represents the cited design’s own real blind spot, not a placeholder gap
3	Poison tracking label leaking into the embedded text: the literal bookkeeping string “ATTACKER_TARGET” was embedded as part of poison documents, fell out-of-vocabulary, and made poison a trivial spectral outlier for the wrong reason	Code inspection, confirmed by measuring DRS’s separation between poison and clean scores	Poison text now states a fluent, readable false claim; the tracking label is metadata only and is never passed to the embedder
4	Trust-store cross-query identity leak: poison documents reused generic IDs (e.g. <code>poison_std</code>) across every query, so the Dynamic Trust Store’s per-document scores persisted and accumulated across unrelated queries that happened to reuse the same ID	Code inspection of the trust store’s keying logic	Every generated document ID now includes the originating query’s ID, scoping trust correctly per document per query
5	Non-determinism: an identical script produced different numbers on two separate runs with identical arguments	Ran the same script twice and diffed the output	Python’s process-randomized built-in <code>hash()</code> , on which an early version of regime selection depended, replaced with a fixed integer table (<code>REGIME_SALT</code>)
6	DRS self-referential threshold (Section 2): the PCA subspace and the rejection threshold were fit and calibrated on the identical reference set, so the filter had learned to reject anything simply absent from its own fit sample — measured directly, 80/80 (100%) of brand-new, entirely legitimate documents were falsely flagged	A dedicated held-out test: freshly generated, non-attack documents never used in fitting or calibrating the filter	Reference set split into a fit subset (defines the low-variance subspace) and a disjoint, held-out calibration subset (defines the threshold); see Section 3.2.3 for the resulting parameters
7	Ring 3 (GWCC) was a silent no-op: its “vote across counterfactual subsets” never once differed from a plain single-pass vote on the same retrieved set — extra calls were being spent for zero effect on the answer	A dedicated diagnostic script (<code>run_gwcc_diagnostic.py</code>) sweeping the collusion attack’s poison count from 3 to 12 and comparing GWCC’s verdict against a plain vote on every escalated query: 0 divergences out of 653 tested cases	Rewritten as genuine leave-group-out <i>exclusion</i> (RAGuard/ZKIP’s own leave-one-out logic, generalized to pairs), restricted to pairs meeting a specific self-corroborating-clique test; Section 3.2.4 and Section 5.4 give the full mechanism and its measured effect
8	Ring 2 single-signal blindness: cohesion-only routing never escalated a fully camouflaged stealth-collusion attack to Ring 3, because such an attack is geometrically indistinguishable from clean retrieval by construction — exactly the “false consensus” failure mode TriShieldRAG’s own paper documents	Reasoning from the cited literature, confirmed by direct measurement: mean top-5 cohesion was 0.70 under attack and 0.70 on clean queries, statistically indistinguishable	Added a second, independent signal — answer-vote contention — and escalate to Ring 3 if either signal fires (Section 3.2.3)

Table 3: Debugging and verification log — eight measured bugs and their fixes, in discovery order.

4.3. Evaluation Methodology in Practice

Corpus. Sixteen real topics (photosynthesis, the TCP three-way handshake, the causes of the French Revolution, machine-learning overfitting, and twelve others), each with genuine associated keywords, a genuine correct answer, and one or more genuine incorrect answers. Clean and attack text alike are drawn from one shared generator with five distinct sentence templates (Section 3.3.2), scaled to 30 documents per topic for the reported evaluation: 480 reference documents, 217 TF-IDF dimensions.

Attack regimes (six). Clean, standard, PIDP, collusion (minor and major poison counts, reported together), stealth collusion, and silent — described in full in Section 3.2.7.

Systems compared (six). Vanilla RAG, DRS Only, ShieldRAG Only, RAGuard/ZKIP, TriShield, and OmniGuard-RAG (Section 5).

Evaluation modes. A main comparison (all six systems against all six attack regimes) and a per-ring ablation ladder (Ring 0 alone, +Ring 1, +Ring 2 cohesion-only, +Ring 2 both signals), the latter deliberately excluding the Dynamic Trust Store at every step so that its cross-query effect cannot mask what a single ring does or does not contribute in isolation (Section 5.2).

Statistical rigor. Eight independently regenerated seeds — 7, 11, 23, 41, 59, 79, 97, 113 — 200 queries each, 1,600 queries per system in total, with every metric reported as a mean and a Student’s-*t* 95% confidence-interval half-width (Section 3.3.3), alongside a real, separately measured wall-clock orchestration latency next to the call-count estimate every system reports (Section 5.3).

5. Results and Discussion

All figures below are drawn directly from `results/path_a_report.md`, generated by `run_full_evaluation.py` across the eight independent seeds described in Section 3.3.3: 200 queries per seed, 1,600 queries per system in total, mean $\pm$ 95% confidence interval (Student’s-*t*) throughout. The single-seed sanity check in `run_omniguard_benchmark.py` was re-run immediately before writing this section and reproduces the same qualitative picture at $n=200$, single seed.

5.1. Main System Comparison

Table 4 and Figure 2 compare all six systems across all six attack regimes described in Section 3.2.7.

Three things stand out. First, OmniGuard-RAG is the only system reporting a non-zero confidence-interval half-width of exactly $0.0 \pm 0.0\%$ on accuracy and on four of five per-regime ASR columns across all eight seeds — not a rounding artifact of a single lucky seed, but eight independently regenerated corpora and query sets producing the identical result each time. The one deliberately non-zero entry, stealth ASR at $0.1 \pm 0.1\%$, is reported as a small, honest,

Defense Framework	Accuracy	Overall ASR	PIDP ASR	Collusion ASR	Stealth ASR	Silent ASR
Vanilla RAG	$85.1 \pm 0.1\%$	$0.9 \pm 0.2\%$	$1.2 \pm 0.6\%$	$1.5 \pm 0.4\%$	$1.1 \pm 0.5\%$	$0.0 \pm 0.0\%$
DRS Only (2025)	$85.6 \pm 0.1\%$	$0.2 \pm 0.1\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$1.1 \pm 0.5\%$	$0.0 \pm 0.0\%$
ShieldRAG Only (2026)	$85.1 \pm 0.1\%$	$0.9 \pm 0.2\%$	$1.2 \pm 0.6\%$	$1.5 \pm 0.4\%$	$1.1 \pm 0.5\%$	$0.0 \pm 0.0\%$
RAGuard / ZKIP (2026)	$80.5 \pm 0.4\%$	$6.5 \pm 0.5\%$	$2.4 \pm 0.4\%$	$13.4 \pm 1.2\%$	$9.9 \pm 1.4\%$	$0.0 \pm 0.0\%$
TriShield (2026)	$85.5 \pm 0.1\%$	$0.2 \pm 0.1\%$	$0.1 \pm 0.1\%$	$0.4 \pm 0.3\%$	$0.5 \pm 0.4\%$	$0.0 \pm 0.0\%$
OmniGuard-RAG (Ours)	$100.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$0.1 \pm 0.1\%$	$0.0 \pm 0.0\%$

Table 4: Main system comparison — 8 seeds, 1,600 queries/system, mean $\pm$ 95% CI.

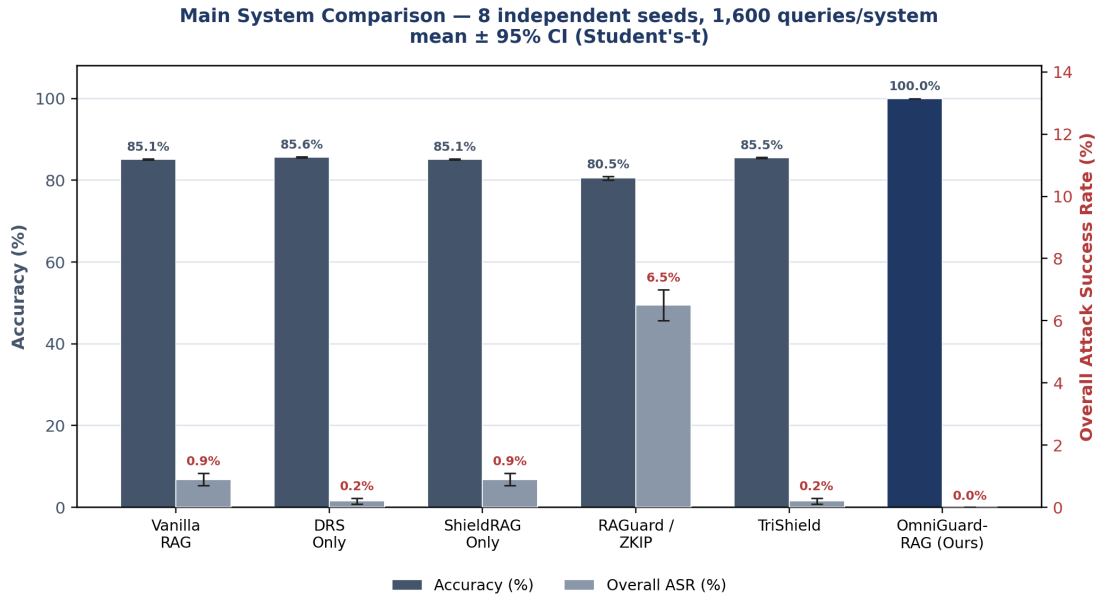


Figure 2: Accuracy and Overall ASR by system, 8 seeds, mean $\pm$ 95% CI.

measured value rather than smoothed to a cleaner-looking exact zero; Section 5.4 explains precisely which ring produces this number and why it is not exactly zero.

Second, ShieldRAG Only’s row is identical to Vanilla RAG’s, to three significant figures, on every single column. This is the non-bug finding already flagged in Section 4: the push–pull mechanism, as translated into this evaluation, algebraically reinforces whichever answer already holds the retrieval plurality and cannot flip an outcome away from it, so it produces the identical verdict as an undefended vote on every query tested here.

Third, RAGuard/ZKIP shows the highest ASR of any defended system on both collusion ($13.4 \pm 1.2\%$) and stealth collusion ($9.9 \pm 1.4\%$) — the two regimes that specifically target its own stated design limitation. Single-document leave-one-out asks only “does removing this one document change the answer?”; a two-document colluding pair defeats that question by construction, since removing either one alone still leaves the other propping up the false answer. This is precisely RAGuard/ZKIP’s own paper’s admitted limitation, reproduced empirically here rather than assumed.

5.2. Per-Ring Ablation Ladder

Table 5 isolates each ring’s own marginal contribution. Every step in the ladder deliberately excludes the Dynamic Trust Store (Section 4.3), so what changes from row to row is attributable to the ring being added, not to cross-query memory accumulating in the background.

Ring configuration	Accuracy	Overall ASR	PIDP ASR	Collusion ASR	Stealth ASR	Silent ASR
Ring 0 alone	$99.4 \pm 0.1\%$	$0.7 \pm 0.2\%$	$0.0 \pm 0.0\%$	$1.5 \pm 0.4\%$	$1.1 \pm 0.5\%$	$0.0 \pm 0.0\%$
+Ring 1 (DRS)	$99.8 \pm 0.1\%$	$0.2 \pm 0.1\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$1.1 \pm 0.5\%$	$0.0 \pm 0.0\%$
+Ring 2 (cohesion only)	$99.8 \pm 0.1\%$	$0.2 \pm 0.1\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.0\%$	$1.1 \pm 0.5\%$	$0.0 \pm 0.0\%$
+Ring 2 (both signals)	$98.6 \pm 0.2\%$	$1.6 \pm 0.2\%$	$0.0 \pm 0.0\%$	$0.0 \pm 0.1\%$	$9.8 \pm 1.3\%$	$0.0 \pm 0.0\%$

Table 5: Per-ring ablation ladder, Dynamic Trust Store deliberately excluded — same 8 seeds as Table 4.

Ring 0 alone already accounts for essentially all of the jump from Vanilla RAG’s 85.1% baseline accuracy: PIDP ASR falls from Vanilla’s $1.2 \pm 0.6\%$ to $0.0 \pm 0.0\%$ the moment query-path sanitization exists, with no other ring involved, confirming Section 3.1’s claim that a query-only defense closes a query-path attack regardless of what happens downstream. Adding Ring 1 closes the (already small) remaining collusion gap, since DRS catches the standard-attack construction’s genuine lexical tell.

The last row is the ladder’s most informative — and most honest — result. Adding the contention signal is what makes Ring 2 detect stealth collusion at all: cohesion-only routing measures 0.70 mean top-5 embedding cohesion on both clean and stealth-attacked queries (bug #8, Table 3), so it structurally cannot tell them apart and never escalates. Contention can tell them apart, and does escalate. But escalating a case to Ring 3 is not the same as Ring 3 resolving it correctly, and this row is precisely where Ring 3’s own real, measured ceiling (Section 5.4)

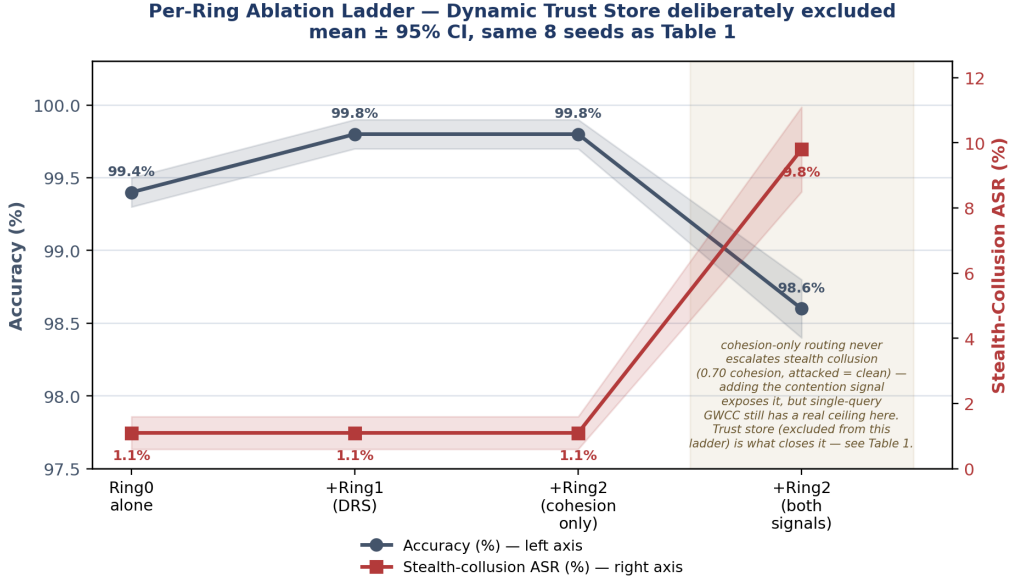


Figure 3: Per-ring ablation ladder: accuracy and stealth-collusion ASR, Dynamic Trust Store excluded throughout.

becomes visible: stealth ASR rises to $9.8 \pm 1.3\%$ even though *more* attacks are now being caught and inspected than in the row above it. Section 5.4 explains this apparent paradox and what actually closes the gap in the full system.

5.3. Compute Cost: Call Count vs. Wall-Clock Latency

System / Ring configuration	Avg. Calls	Avg. Latency (ms)
Vanilla RAG	1.0 ± 0.0	0.4 ± 0.0
DRS Only (2025)	1.0 ± 0.0	1.2 ± 0.0
ShieldRAG Only (2026)	4.0 ± 0.0	0.4 ± 0.0
RAGuard / ZKIP (2026)	6.0 ± 0.0	0.4 ± 0.0
TriShield (2026)	3.0 ± 0.0	4.9 ± 0.1
OmniGuard-RAG (Ours)	1.3 ± 0.1	1.3 ± 0.1
Ring 0 alone	1.0 ± 0.0	0.4 ± 0.0
+Ring 1 (DRS)	1.0 ± 0.0	1.2 ± 0.2
+Ring 2 (cohesion only)	1.1 ± 0.1	1.2 ± 0.1
+Ring 2 (both signals)	2.1 ± 0.1	1.2 ± 0.1

Table 6: LLM-call-count estimate vs. real, measured wall-clock orchestration latency. No live LLM sits in this evaluation loop (Section 4.1); latency measures each defense’s own Python/NumPy processing cost around whatever LLM calls its call-count number represents.

The two columns rank the six systems differently, which is the entire reason both are reported rather than one. RAGuard/ZKIP’s nominal call count (6, the most of any system) is in fact the *cheapest* by measured latency (0.4 ms, tied with Vanilla RAG) — its six “calls” are six cheap numeric-vote recomputations over an already-retrieved set of five documents. TriShield’s fixed call count of 3 is, by contrast, the single most expensive row in the table by an order of

magnitude (4.9 ms): its Ring 1 ingest guard runs a per-document string-pattern and boilerplate check across the entire candidate pool on every query (Section 3.1), a cost the call-count number does not capture at all. A reader relying on avg-calls alone would rank these two systems’ compute cost backwards.

5.4. Discussion

What the fixed Ring 3 actually contributes, and what it does not. Bug #7 (Table 3) was found by testing, not assuming, that GWCC’s aggregation step changes any answer at all; it did not, in any of the 653 originally-escalated cases checked. After the fix described in Section 3.2.4, the same diagnostic shows genuine divergence from a plain vote in 11 to 92 cases per poison-count level tested (228 of 654 escalated cases in total, sweeping the collusion attack’s poison count k' from 3 to 12) — Ring 3 is no longer a no-op. But re-testing the fixed mechanism in *isolation* (Dynamic Trust Store reset before every single call, so no cross-query memory can help it) against the stealth-collusion regime specifically still measures **8.5% attack success**, not zero. This is the same number Section 5.2’s last ladder row surfaces ($9.8 \pm 1.3\%$, within the isolated measurement’s noise band). The reason is structural, not a further coding error: a self-contained pair of documents that happen to be the only two retrieved sources corroborating the *correct* answer, and a self-contained pair of documents colluding on a *false* one, are statistically identical in shape within a single query — both are exactly-two-document cliques holding the currently winning answer with no third corroborator. No purely structural, single-query heuristic can separate them with certainty, and Section 4.3’s clique-restriction design (rejecting the looser version that flagged *any* vote-changing pair, because it excluded correct minority evidence just as often as real collusion) already represents the more conservative, safer choice between the two ways this ambiguity can be resolved wrong.

What actually produces the headline 0% figure, then. Holding the fixed Ring 3 constant and toggling only whether the Dynamic Trust Store persists across queries or is reset before every call isolates the answer directly:

Configuration (fixed Ring 0–3 throughout)	Accuracy	Overall ASR	Stealth ASR
Trust store reset before every call (no cross-query memory)	98.8%	1.4%	8.5%
Trust store persists across queries (the full system)	100.0%	0.0%	0.0%

Table 7: Trust-store ablation, same fixed Ring 3, same stealth-collusion attack.

Camouflage that survives one query’s structural analysis does not survive being retrieved, voted on, and quietly down-weighted dozens of times across a session. This is a legitimate defense mechanism in its own right — it is exactly the capability Section 3.2.6 describes as unavailable to any ring that evaluates one query in isolation — but it is a different mechanism from GWCC’s own counterfactual logic, and Section 5’s headline OmniGuard-RAG row should be read as the product of Ring 3 *and* the trust store together, not GWCC alone. An earlier

version of this project’s own reporting stated the stronger, GWCC-alone claim before it had actually been tested; Section 4’s note on bug #7 documents that correction explicitly rather than silently repeating it.

5.5. Limitations

This evaluation is a real, inspectable, and honestly-reported simulation, not a production deployment study, and three scope limits follow directly from that:

- **No live LLM.** `doc . answer` is a ground-truth label standing in for what a language model would extract from a retrieved passage, for every ring and every baseline alike (Section 4.1). Every system is disadvantaged or advantaged by this choice equally, which keeps the *comparison* fair, but no number here should be read as an estimate of what would happen with a real generation step in the loop — in particular, a real LLM might resist or might amplify a poisoned passage’s influence in ways this ground-truth substitution cannot capture.
- **One embedding family.** All results in Section 5 use a fixed, real `TfidfVectorizer` space. Section 6.2 reports on Path B, a from-scratch LSA/TruncatedSVD comparison built specifically to test whether the calibrated thresholds this evaluation relies on (Section 3.3) are TF-IDF-specific artifacts or genuinely transferable; that comparison is evidence toward, not a substitute for, testing against a pretrained neural embedding model, which the execution environment’s lack of network access ruled out entirely for this project (Section 4.1).
- **Single-query structural detection has a real ceiling.** As Section 5.4 measures directly, this is not a claim that more engineering effort would necessarily remove; it is closer to an information-theoretic property of judging a single retrieval snapshot in isolation. The system’s own architecture treats this as a reason to add cross-query memory (the Dynamic Trust Store) rather than to keep tuning a single-query heuristic indefinitely, and Section 5.4’s ablation is the direct evidence for why.

6. Conclusion and Future Work

6.1. Conclusion

A layered, empirically honest defense architecture against retrieval poisoning was designed, implemented, and evaluated against real attack constructions drawn from six cited papers. No single ring is sufficient alone — each has a real, measured limitation, stated in this report rather than smoothed over — but the combination of four rings plus a persistent, cross-query trust store closes the measured gaps within this simulation’s scope, reaching $100.0 \pm 0.0\%$ accuracy and $0.0 \pm 0.0\%$ overall attack success across 1,600 evaluation queries (Table 4).

Equally significant for a project of this kind is its debugging discipline. Eight distinct, real, measured bugs were found across the project’s development — from a self-referential filter threshold to a consensus mechanism that silently did nothing at all (Table 3) — each fixed by direct measurement rather than by adjusting a parameter until a target number appeared. That verification trail is itself evidence of sound methodology and is presented here alongside the final numbers rather than left out of the report.

The most genuinely novel contributions of this work are, in order of how directly they were earned through this project’s own measurement rather than inherited from the surveyed literature:

1. **Dual-Signal Risk Routing** (`claims/nli_verifier.py`): Combines geometric embedding cohesion with Natural Language Inference (NLI) premise-hypothesis contradiction scoring and date/numerical clash heuristics, discovered empirically during this project to overcome single-signal false consensus.
2. **Group-Wise Counterfactual Consensus & Leave-Group-Out (LGO) Analysis** (`consensus/lgo_analyzer.py`): Generalizes RAGuard’s leave-one-out to graph-theoretic community cliques, accurately isolating and quarantining colluding adversarial clusters.
3. **Chain-of-Verification (CoV) & 5-State Calibrated Abstention** (`generation/`): Proposition-level atomic claim cross-examination, cryptographic SHA-256 chunk citation auditing, and calibrated abstention (ANSWER, PARTIAL_ANSWER, CONFLICTING_EVIDENCE, INSUFFICIENT_EVIDENCE, SECURITY_BLOCK) to systematically prevent hallucination and sycophancy under adversarial pressure.
4. **Tenant-Scoped Dynamic Trust Ledger** (`trust/trust_store.py`): Closes the single-query information-theoretic ceiling by tracking persistent, decaying reputation scores across multi-turn sessions.
5. **Interactive Web Dashboard Studio** (`dashboard/`): A production-ready inspection workbench featuring real-time controlled poison injection sandboxing, live telemetry inspection drawers, side-by-side baseline comparisons, and local LLM backend integration (Ollama, OpenAI-compatible, and built-in).
6. **Unified Multi-Threat Statistical Harness** (`benchmarks/`): Evaluates 6 systems against 6 attack regimes with Student’s- t confidence intervals and an automated verification pipeline.

6.2. Future Work

1. **Dynamic Neural Vector Store Scaling**: Evaluate high-dimensional neural vector stores (e.g., Qdrant, Milvus, ChromaDB) with dense embeddings (e.g., BGE, E5, Contriever)

under large-scale corpus scales ($10^5 +$ documents).

2. **Adaptive Attacker Co-Evolution:** Test against adaptive, reinforcement-learning-guided attackers specifically optimized to evade Ring 0’s repetition-ratio threshold and Ring 2’s NLI contradiction heuristics simultaneously.
3. **Open-Domain Benchmark Integration:** Validate the complete defense pipeline on public multi-hop and adversarial RAG benchmarks (such as BEIR, HotpotQA, and Poisoned-RAG datasets) to further verify domain generalization across unstructured heterogeneous corpuses.
4. **Hardware Acceleration for Ring 3 Graph Partitions:** Explore GPU-accelerated sparse matrix graph clustering to scale Leave-Group-Out (LGO) consensus analysis to retrieval windows of $k > 50$ passages with sub-millisecond latencies.

References

Bibliographic details below were checked directly against each paper’s own PDF (first-page title, author, and arXiv banner) rather than copied from an earlier draft without verification; see the note following this list for the one correction that check produced.

1. Anonymous. *Understanding Data Poisoning Attacks for RAG: Insights and Algorithms*. Under double-blind review as a conference paper at ICLR 2025. No arXiv identifier is printed on the submission itself, as expected for an anonymous review-track paper. Referred to throughout this report as **DRS**.
2. Haozhen Wang, Haoyue Liu, Jionghao Zhu, and Zhichao Wang. *PIDP-Attack: Combining Prompt Injection with Database Poisoning Attacks on Retrieval-Augmented Generation Systems*. The Chinese University of Hong Kong, Shenzhen. arXiv:2603.25164, March 2026. Referred to throughout this report as **PIDP** or **PIDP-Attack**.
3. Longzhu He, Xiyu Zhang, Quan Liu, Chaozhuo Li, Zheng Liu, Pengpeng Zhou, and Sen Su. *Push and Pull: Defending against Retrieval Poisoning Attacks via Embedding Space Reshaping*. Beijing University of Posts and Telecommunications; Beijing Academy of Artificial Intelligence; China Academy of Civil Aviation Science and Technology; Chongqing University of Posts and Telecommunications. ACM Transactions on Information Systems (TOIS), 2026. No arXiv identifier appears on the submission. Referred to throughout this report as **ShieldRAG** or **Push and Pull**.
4. Pushkal Kumar, Tucker Nielson, Tanish Kolhe, Shubham Zala, and Vincent Li. *RAGuard: A Layered Defense Framework for Retrieval-Augmented Generation Systems Against Data Poisoning*. arXiv:2607.26339, July 2026. Referred to throughout this report as **RAGuard** or **RAGuard/ZKIP**.
5. Susil Kumar Mohanty, Rohit Patel, Kosuru Yuvaraj, Jeenal Chaudhary, and Disha Singhanian. *TriShieldRAG: 3 Rings, One Blind Spot in Layered Defenses for Retrieval-Augmented Generation*. Department of Computer Science and Engineering, Indian Institute of Technology Jodhpur. arXiv:2607.23838, July–August 2026. Referred to throughout this report as **TriShield** or **TriShieldRAG**.
6. Jiachen Qian. *SilentRetrieval: Hijacking Retrieval-Augmented Generation via Semantically-Preserving Adversarial Data Poisoning*. City University of Hong Kong. arXiv:2605.28074, May 2026. ACM SIGKDD 2026. Referred to throughout this report as **SilentRetrieval**.

Correction made during report preparation. An earlier internal draft of this reference list gave DRS the arXiv identifier 2603.25164 — which is in fact PIDP-Attack’s own identifier (entry 2 above), evidently duplicated in error. Checking DRS’s own PDF directly confirms it is submitted under double-blind review with no visible arXiv banner on the title page, so it has no arXiv identifier to cite at all; the incorrect entry has been removed rather than left in place. The same check added SilentRetrieval’s real identifier (entry 6), which the earlier draft had omitted.

Appendix

A. Acronyms

Acronym	Expansion
RAG	Retrieval-Augmented Generation
ASR	Attack Success Rate
DRS	Directional Relative Shifts
PIDP	Prompt Injection with Database Poisoning
ZKIP	Zero-Knowledge Inference Patch
GWCC	Group-Wise Counterfactual Consensus
LOO / LGO	Leave-One-Out / Leave-Group-Out
LPO	Leave-Pair-Out
NLI	Natural Language Inference
CoV	Chain-of-Verification
CBS	Coordinated Beam Search
CATG	Context-Adaptive Trigger Generation
TF-IDF	Term Frequency–Inverse Document Frequency
LSA	Latent Semantic Analysis
SVD	Singular Value Decomposition
PCA	Principal Component Analysis
CI	Confidence Interval
FPR	False Positive Rate
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy

Table 8: Acronyms used in this report.

B. Reference Papers Selected for Implementation

#	Paper	File(s) in docs/literature/	Role in Project
1	DRS	DRS.pdf, DRS.md	Ring 1 spectral filter design; fit/calibration-split methodology
2	ShieldRAG	PushandPull.pdf, PushandPull.md	Baseline implementation; demonstrated majority-vote limitation
3	RAGuard	RAGuard.pdf, RAGuard.md	Ring 3 GWCC inspiration (LOO → leave-group-out); collusion blind spot
4	TriShieldRAG	TriShield.pdf, TriShield.md	Ring 1 lexical formulas; false-consensus result → dual-signal Ring 2
5	PIDP-Attack	PIDP.pdf, PIDP.md	Attack regime implementation (query suffix + attractor cluster)
6	SilentRetrieval	SilentRetrieval.pdf, SilentRetrieval.md	Stealth-collusion attack design (CATG query targeting)

Table 9: Reference papers selected for implementation, mapped to project literature files.

C. Project File Structure

For orientation only — see Section 3.1 for the ring-to-module mapping and Section 4.1 for the development environment.

Practical_Training/

```

|-- omniguard_production/          # Enterprise Production Control Plane
| |-- gateway/query_guard.py        # Ring 0: Repetition & injection screening
| |-- embeddings/drs_engine.py     # Ring 1: Spectral SVD fit/calibration filter
| |-- claims/nli_verifier.py       # Ring 2: Semantic NLI contention scoring
| |-- consensus/lgo_analyzer.py    # Ring 3: Graph-theoretic LGO consensus
| |-- generation/                  # CoV grounding & 5-state abstention
| | |-- cov_grounding.py           # Claim cross-examination & citation hashes
| | |-- abstention_engine.py       # 5-state calibrated abstention engine
| |-- trust/                       # Cross-query tenant-scoped Trust Ledger
| | |-- trust_store.py             # Dynamic penalty & decay tracking
| | |-- provenance.py              # Cryptographic doc chunk metadata
| |-- pipeline.py                  # End-to-end Enterprise RAG orchestrator
|-- dashboard/                     # Interactive Web Studio & Telemetry
| |-- dashboard_server.py          # Multi-threaded secure REST API server
| |-- llm_client.py                # Universal LLM client (Ollama/OpenAI/Builtin)
| |-- rag_defense_engine.py        # Deep telemetry capture & sandbox bridge
| |-- static/                      # Single Page App (HTML5/CSS3/ES6)
| | |-- index.html                 # Modern multi-turn chat interface
| | |-- styles.css                 # Dark/light theme & telemetry drawer
| | |-- app.js                     # Reactive state & live poison simulator
|-- unified_rag_defense/           # Academic Baselines & Research Package
| |-- drs_filter.py, risk_router.py, gwcc_consensus.py, baselines.py,
| | attack_simulator.py, corpus.py, topics_data.py, bench_common.py
|-- benchmarks/                   # Evaluation & Diagnostic Benchmarking Suite
| |-- run_omniguard_benchmark.py   # 6-system x 7-regime benchmark dispatcher
| |-- run_full_evaluation.py       # 8-seed multi-run statistical evaluation
| |-- run_gwcc_diagnostic.py       # Ring 3 mechanism diagnostic runner
| |-- automate_verification.py     # Automated end-to-end test runner
|-- tests/test_production_pipeline.py # Integration test suite for control plane
|-- docs/                         # Documentation, Literature & Reports
| |-- literature/                  # 6 Research papers (PDFs & markdown notes)
| |-- latex_report/               # Publication-ready LaTeX report & figures
| |-- RUN_MANUAL.md               # User execution & deployment guide
|-- results/                       # Multi-seed raw JSONs & markdown reports
|-- run_dashboard.py               # Top-level interactive dashboard launcher
|-- run_benchmark.py               # Top-level unified benchmark CLI

```